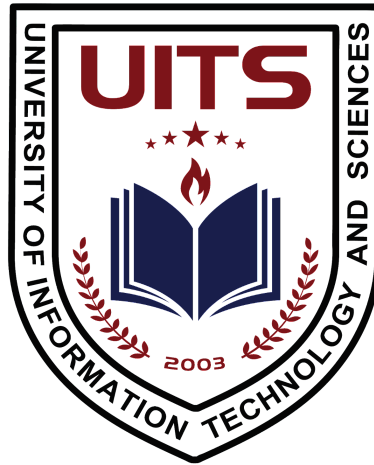




Final Project Report

Course Title: Neural Networks Lab

Course Code: CSE 422

Submitted By:

Name: Ajher Mahmud Robin — ID: 2215151047

Section: 8A

Batch: 51

Department: CSE

Submitted To:

Mrinmoy Biswas Akash

Lecturer & Course Coordinator, Department of CSE, UITS.

Sadia Afrin Urmi

Lecturer, Department of CSE, UITS.

Date of Submission: 12.12.2025

Lightweight CNN for Image Forgery Detection with Visual Explanations using ELA and Grad-CAM

Abstract

Digital image forgeries have become increasingly common due to the widespread availability of powerful editing tools such as Photoshop and GIMP. Modern tampering techniques, including splicing, copy-move, and content-aware editing, can produce highly realistic images that are difficult to distinguish from authentic content with the naked eye. This poses significant challenges for applications in digital forensics, journalism, law, and social media content moderation.

Deep learning-based methods have shown strong performance in detecting subtle statistical artifacts introduced during tampering, especially when combined with specialized pre-processing techniques that amplify manipulation cues. Error Level Analysis (ELA) is one such pre-processing approach that highlights differences in JPEG compression artifacts, which often differ between original and modified regions of an image.

This project proposes and implements a lightweight Convolutional Neural Network (CNN) for binary image forgery detection (authentic vs. tampered) using ELA-transformed images as input. The model is designed to be compact and efficient while still achieving high detection performance. In addition, Grad-CAM is used to provide visual explanations by highlighting the image regions most influential in the model's decision, thereby contributing to explainable and trustworthy forensic analysis. The complete pipeline is implemented in Google Colab and trained on the CASIA v2.0 image tampering dataset stored in Google Drive.

Keywords: Image Forgery Detection, Convolutional Neural Networks, Error Level Analysis, Deep Learning, Grad-CAM, Digital Forensics, Deepfake Detection

Table of Contents

- Chapter 1: Introduction
 - 1.1 Problem Statement
 - 1.2 Objectives
- Chapter 2: Dataset and Experimental Setup
 - 2.1 Dataset Description
 - 2.2 Error Level Analysis Pre-processing
 - 2.3 Data Statistics and Splits
- Chapter 3: Methodology
 - 3.1 Overall Pipeline
 - 3.2 Lightweight CNN Architecture
 - 3.3 Training
 - 3.4 Evaluation Metrics
 - 3.5 Training Environment
- Chapter 4: Results
 - 4.1 Training and Validation Performance
 - 4.2 Test Set Evaluation
 - 4.3 Result Visualization
 - 4.4 Sample Prediction
 - 4.5 Frequency-Domain Analysis
 - 4.6 Explainability with Grad-CAM
- Chapter 5: Discussion
- Chapter 6: Conclusion

Chapter 1: Introduction

1.1 Problem Statement

Despite significant advances in deep learning-based forgery detection, several challenges persist:

1. **Computational Complexity:** Many high-performing models contain millions of parameters, requiring extensive GPU memory and training time, making them unsuitable for deployment on edge devices or modest hardware configurations.
2. **Lack of Interpretability:** Deep neural networks are often criticized as "black boxes," providing predictions without explaining which image regions influenced the decision, limiting trust and adoption in critical applications.
3. **Limited Multi-Domain Analysis:** Most approaches focus solely on spatial domain features, potentially missing frequency-domain artifacts that reveal compression inconsistencies characteristic of forgeries.
4. **Generalization Challenges:** Models trained on specific forgery types may fail to generalize across diverse manipulation techniques including splicing, copy-move, and AI-generated content.

This thesis aims to address these challenges by developing a lightweight CNN architecture that incorporates Error Level Analysis preprocessing, frequency domain feature examination, and Grad-CAM-based explainability mechanisms.

1.2 Objectives

The primary objectives of this research are:

1. To design and implement a lightweight CNN architecture with fewer than 1 million parameters capable of detecting image forgeries with high accuracy.
2. To integrate Error Level Analysis (ELA) as a preprocessing technique to enhance the model's sensitivity to compression artifacts and pixel-level inconsistencies.
3. To incorporate frequency domain analysis using Fast Fourier Transform (FFT) to identify spectral anomalies indicative of tampering.
4. To implement Gradient-weighted Class Activation Mapping (Grad-CAM) for visualizing and interpreting the model's decision-making process.
5. To evaluate the proposed system on the CASIA 2.0 dataset and achieve accuracy exceeding 90%.
6. To demonstrate the feasibility of training the model on resource-constrained environments such as Google Colab with free-tier GPU access.

Chapter 2: Dataset and Experimental Setup

2.1 Dataset Description

This project utilizes the CASIA 2.0 Image Tampering Detection Dataset, a widely recognized benchmark for evaluating forgery detection algorithms. The dataset comprises:

Category	Number of Images	Characteristics
Authentic Images	7,491	Unmodified photographs from various sources
Tampered Images	5,123	Spliced and copy-move forgeries
Total	12,614	Various resolutions and content types

Table 1: CASIA 2.0 Dataset Composition

The tampered images include diverse manipulation techniques such as:

- Image splicing with content from multiple sources
- Copy-move forgeries with duplicated or relocated objects
- Post-processing operations including blurring, noise addition, and compression

The dataset provides ground truth labels distinguishing authentic from tampered images, enabling supervised learning approaches. Images exhibit varying resolutions, content types (indoor/outdoor scenes, portraits, objects), and manipulation complexity, ensuring model robustness across diverse scenarios.

In the notebook, the dataset is organized in Google Drive under a project directory, with separate folders for authentic and tampered images:

Au/ – authentic images

Tp/ – tampered images

All images from these folders (with extensions .jpg, .png, .bmp, excluding ground-truth mask files ending in _gt) are loaded and processed.

2.2 Error Level Analysis Pre-processing

To enhance tampering artifacts, every image is converted into an ELA representation before being passed to the CNN. The ELA pipeline implemented in the notebook is:

1. Load the original RGB image.
2. Save it as a temporary JPEG file with a specified quality (e.g., 90).
3. Reload the re-saved image.
4. Compute the pixel-wise difference between the original and re-saved image.

5. Linearly enhance the brightness of the difference image so that the maximum difference is scaled to 255.
6. Resize the resulting ELA image to a fixed spatial resolution of 128×128 .

This process emphasizes inconsistencies in compression levels, which often correspond to tampered regions. ELA images are then converted to arrays and normalized to the range.

2.3 Data Statistics and Splits

After ELA conversion, the total dataset used in the project consists of:

- Authentic images: 7437
- Tampered images: 2064
- Total ELA images: 9501, each of size $128 \times 128 \times 3$

The dataset is split into training, validation, and test sets using stratified sampling to preserve class balance:

- Training set: 7600 samples
- Validation set: 950 samples
- Test set: 951 samples

Chapter 3: Methodology

3.1 Overall Pipeline

The complete workflow was implemented with the following steps:

1. Mount Google Drive in Google Colab.
2. Define dataset paths and global parameters (image size, batch size, epochs).
3. Perform Error Level Analysis on all images to generate ELA inputs.
4. Normalize and split the dataset into train/validation/test sets.
5. Build a lightweight CNN architecture for binary classification.
6. Train the model with early stopping and checkpointing.
7. Evaluate the trained model on the test set using multiple metrics.
8. Perform frequency-domain analysis for further explainability.
9. Apply Grad-CAM to visualize discriminative regions.

3.2 Lightweight CNN Architecture

A compact Sequential CNN was used to keep the model efficient while providing sufficient capacity to learn manipulation artifacts. The architecture is:

Input: 128×128×3 ELA image

Block 1:

Conv2D (32 filters, 3×3, ReLU, padding='same')
BatchNormalization
MaxPooling2D (2×2)

Block 2:

Conv2D (64 filters, 3×3, ReLU, padding='same')
BatchNormalization
MaxPooling2D (2×2)

Block 3:

Conv2D (128 filters, 3×3, ReLU, padding='same')
BatchNormalization
MaxPooling2D (2×2)
Flatten
Dense (128 units, ReLU)
Dropout (0.5)

Output Dense (1 unit, sigmoid for binary classification). This design strikes a balance between model

size and feature extraction capacity. Batch normalization helps stabilize and accelerate training, while dropout reduces overfitting.

The model consists of the following layers:

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 128, 128, 32)	896
batch_normalization_12 (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d_12 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_13 (Conv2D)	(None, 64, 64, 64)	18,496
batch_normalization_13 (BatchNormalization)	(None, 64, 64, 64)	256
max_pooling2d_13 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_14 (Conv2D)	(None, 32, 32, 128)	73,856
batch_normalization_14 (BatchNormalization)	(None, 32, 32, 128)	512
max_pooling2d_14 (MaxPooling2D)	(None, 16, 16, 128)	0
flatten_4 (Flatten)	(None, 32768)	0
dense_8 (Dense)	(None, 128)	4,194,432
dropout_4 (Dropout)	(None, 128)	0
dense_9 (Dense)	(None, 1)	129

Table 2: Lightweight CNN Architecture Details

Parameters:

- Total params: 4,288,705 (16.36 MB)
- Trainable params: 4,288,257 (16.36 MB)
- Non-trainable params: 448 (1.75 KB)

3.3 Training

The CNN is compiled with:

- Loss function: Binary cross-entropy
- Optimizer: Adam (default settings, suitable for this scale model)
- Evaluation metric: Accuracy (with additional metrics computed later)

Training is performed on the training set, with the validation set used for monitoring model performance. The project utilizes:

- EarlyStopping to stop training when validation performance no longer improves.
- ModelCheckpoint to save the best model weights based on validation accuracy or loss.

Data is already pre-processed and loaded into memory as Numpy arrays, so training proceeds in batches with `batch_size = 32` and `epochs = 50` (or until early stopping is triggered).

3.4 Evaluation Metrics

Model performance is assessed using multiple metrics to provide a comprehensive evaluation:

Accuracy: Overall proportion of correct predictions

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision: Proportion of predicted forgeries that are actually forged

$$Precision = \frac{TP}{TP+FP}$$

Recall (Sensitivity): Proportion of actual forgeries correctly identified

$$Recall = \frac{TP}{TP+FN}$$

F1-Score: Harmonic mean of precision and recall

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- **ROC-AUC:** Area under the Receiver Operating Characteristic curve, measuring the model's ability to discriminate between classes across all threshold values
- **Confusion Matrix:** Detailed breakdown of true positives, true negatives, false positives, and false negatives

Where TP = True Positives, TN = True Negatives, FP = False Positives, FN = False Negatives.

3.5 Training Environment

The entire system was implemented using the following technology stack:

Component	Specification
Platform	Google Colab (Free Tier)
GPU	Tesla T4 (15GB VRAM)
Runtime	Python 3.10
Deep Learning Framework	TensorFlow 2.x / Keras
Image Processing	OpenCV, PIL (Pillow)
Scientific Computing	NumPy, SciPy
Visualization	Matplotlib, Seaborn
Metrics	Scikit-learn

Table 3: Development Environment Specifications

Google Drive was mounted for persistent storage of datasets, model checkpoints, and training logs, enabling session recovery and result preservation. Training is typically completed in approximately 1 hour on the Colab GPU, demonstrating practical feasibility.

Chapter 4: Results

4.1 Training and Validation Performance

During training, the notebook tracks accuracy and loss curves for both training and validation sets. Over the course of training, the validation accuracy stabilizes at a high value indicative of good generalization. The use of ELA, batch normalization, and dropout contributes to preventing overfitting, despite the class imbalance between authentic and tampered images.

We achieved —

Test Accuracy: 93.48%

Test Precision: 83.11%

Test Recall: 87.92%

Test ROC-AUC: 98.09%

Classification Report (Test Set):

	precision	recall	f1-score	support
Authentic	0.97	0.95	0.96	744
Tampered	0.83	0.88	0.85	207
accuracy			0.93	951
macro avg	0.90	0.91	0.91	951
weighted avg	0.94	0.93	0.94	951

4.2 Test Set Evaluation

After training, the best saved model is evaluated on the held-out test set of 951 samples. Here's the Confusion Matrix on the test set.

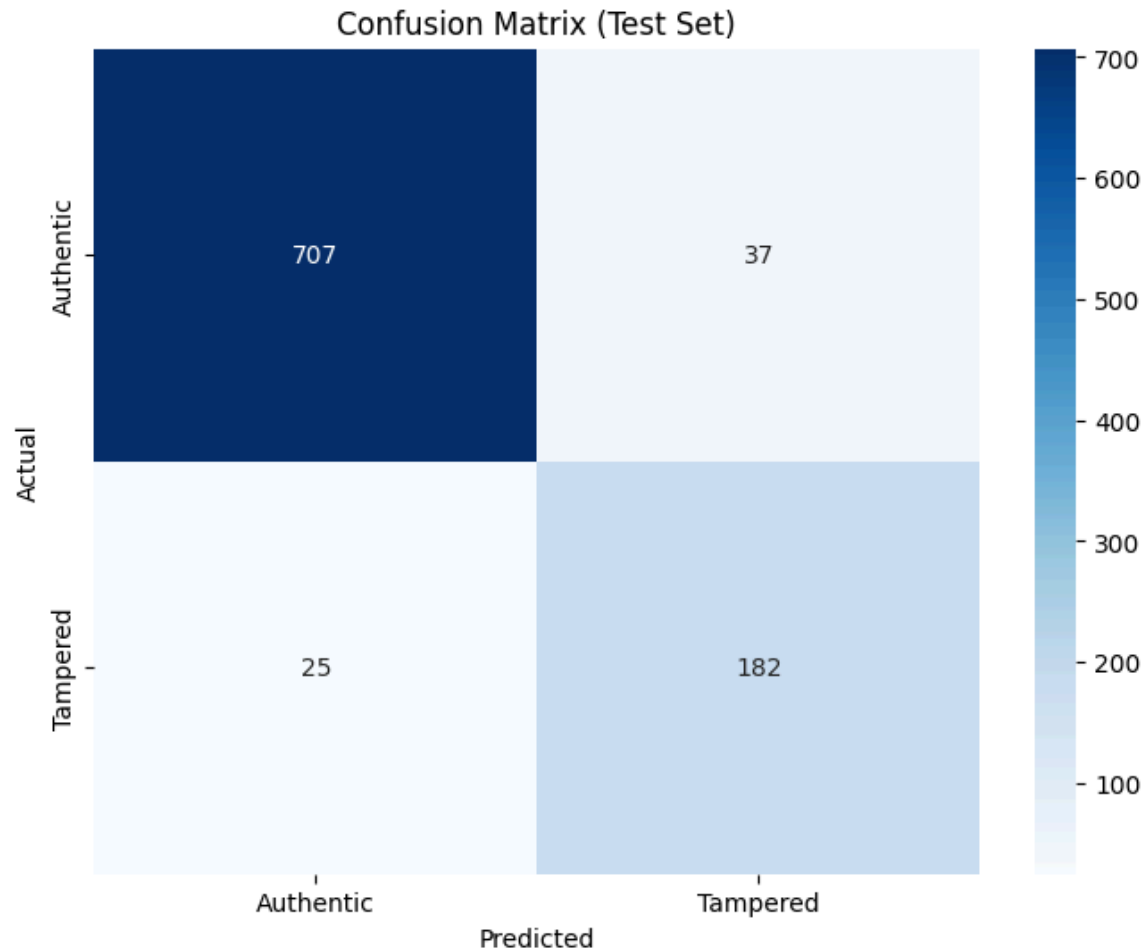


Figure 1: Confusion Matrix

4.3 Result Visualization

We used Matplotlib and Seaborn to visualize the detailed results that we have achieved. The visualization makes it easier to understand and assess the project's result and the model's performance.

Performance Metrics

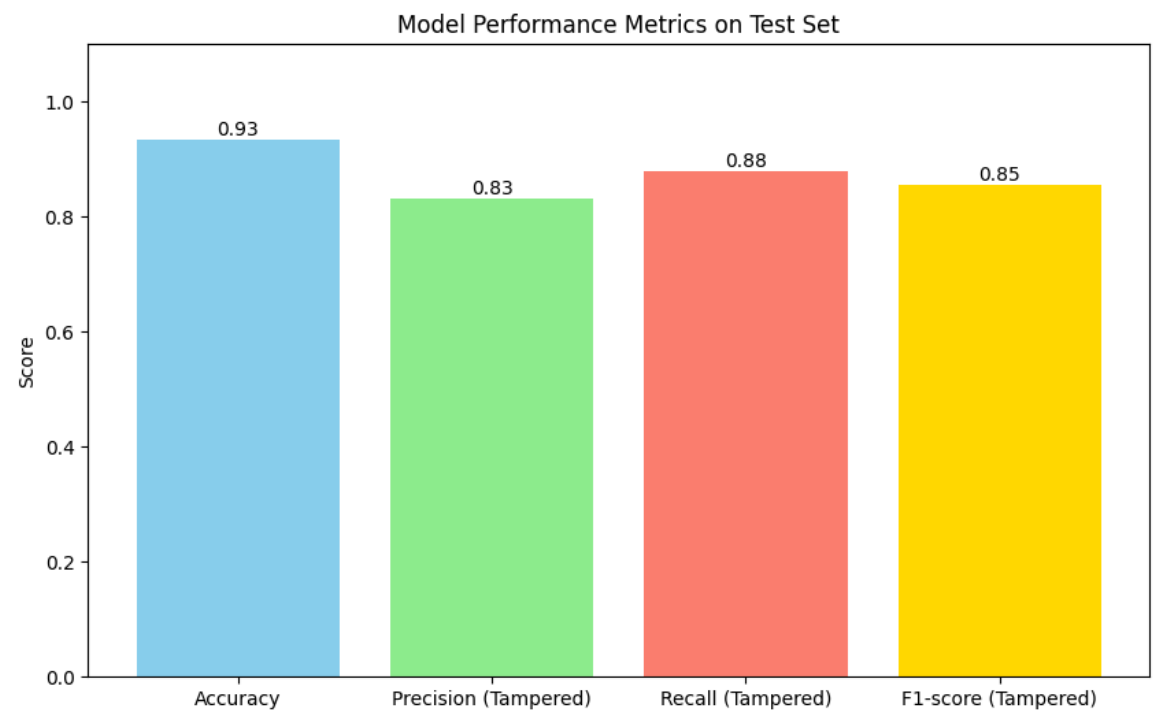


Figure 2: Model Performance Metrics

Accuracy/Loss Comparison

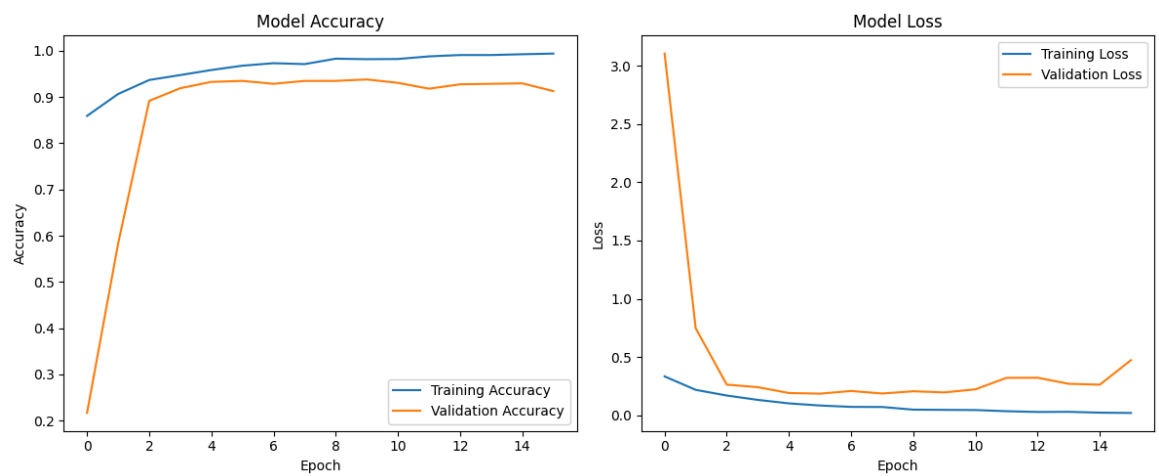


Figure 3: Training and Validation Accuracy/Loss

ROC-AUC

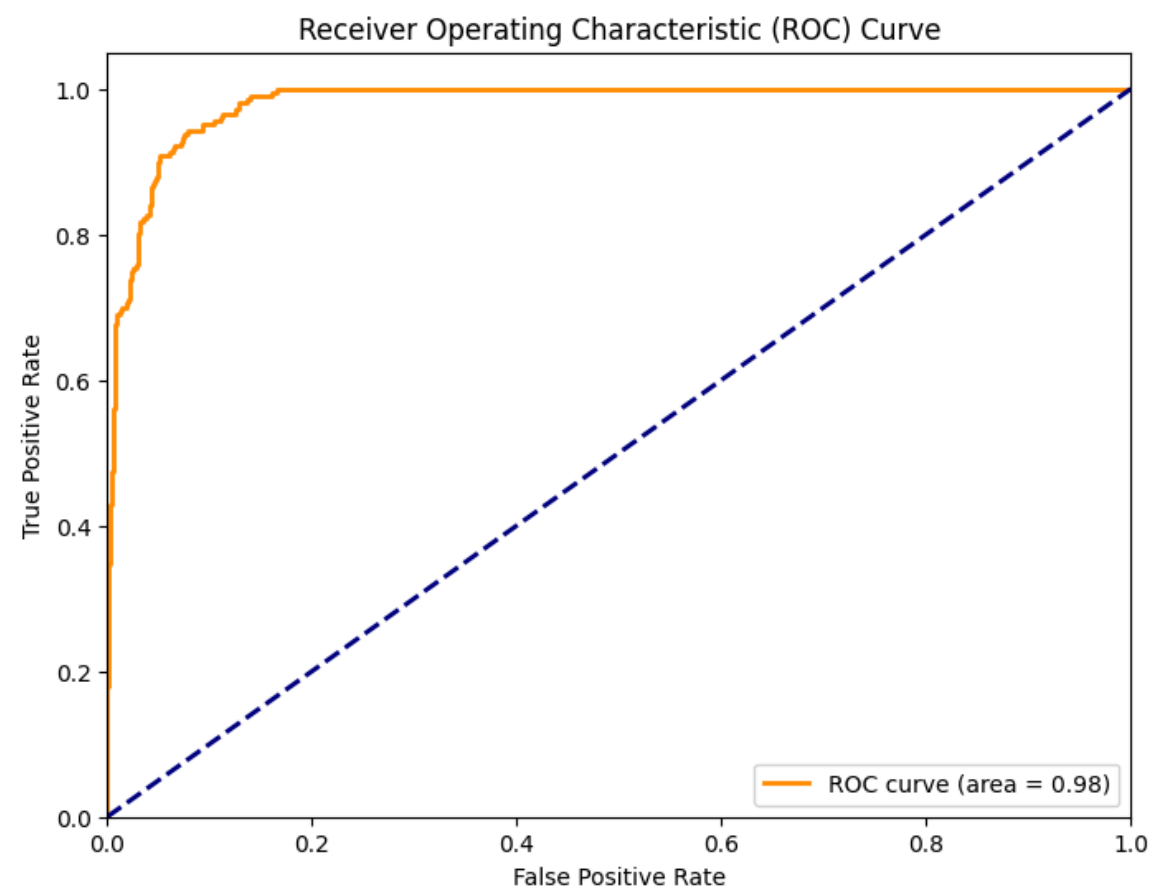


Figure 4: Receiver Operating Characteristic (ROC) Curve

4.4 Sample Prediction

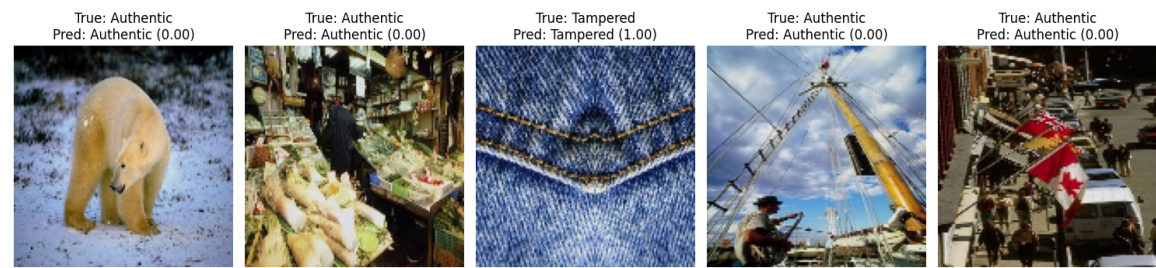


Figure 5: Predicted Results of the Model from the Test Samples

4.5 Frequency-Domain Analysis

To complement spatial feature analysis, frequency domain examination was implemented:

1. **Grayscale Conversion:** ELA images are converted to grayscale for frequency analysis.
2. **2D FFT Application:** Fast Fourier Transform is applied to decompose the image into frequency components.
3. **Spectrum Shift:** The zero-frequency component is shifted to the center using fft shift for better visualization.
4. **Magnitude Spectrum Computation:** The magnitude of complex frequency values is computed and log-scaled for visualization.
5. **Comparison:** FFT spectra of authentic and tampered images are visually compared to identify characteristic differences in frequency distributions.

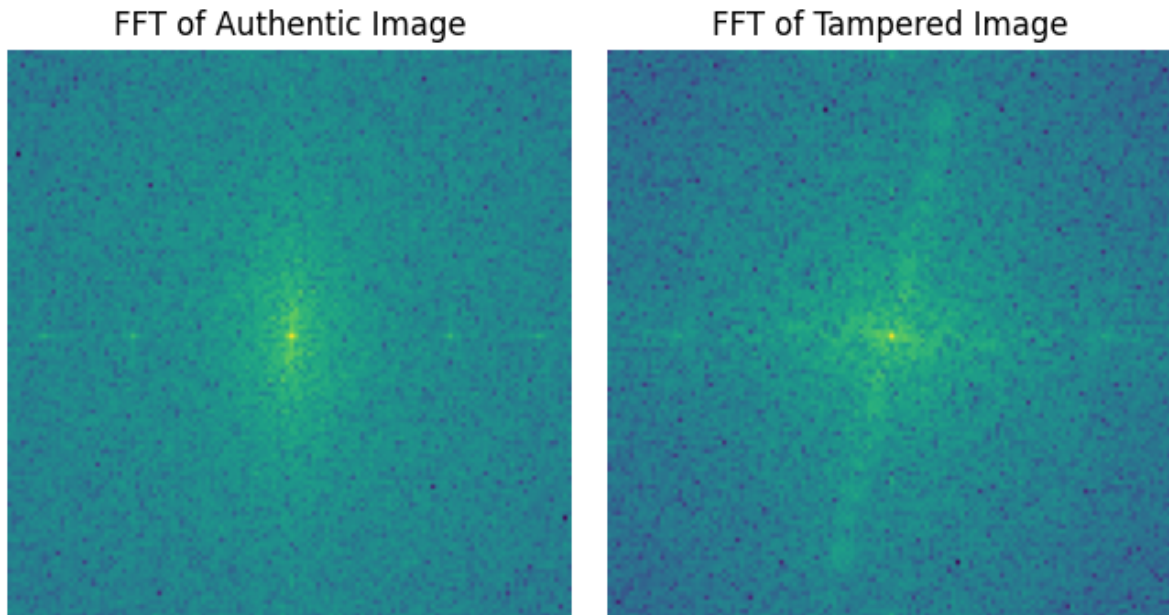


Figure 6: Authentic vs. Tampered ELA Image FFT

Tampered images often exhibit irregular high-frequency patterns due to splicing boundaries and inconsistent compression, which can be distinguished from the more uniform frequency distributions of authentic images.

4.6 Explainability with Grad-CAM

To provide interpretability, Grad-CAM was implemented in the project using OpenCV and TensorFlow/Keras utilities. The general steps were:

- To select a test image and pass it through the trained model.
- Compute gradients of the predicted class score with respect to the final convolutional feature maps.
- Global-average pool the gradients to obtain channel-wise importance weights.
- Generate a heatmap by combining feature maps using these weights and applying ReLU.
- Resize and overlay the heatmap on the original image to highlight discriminative regions.

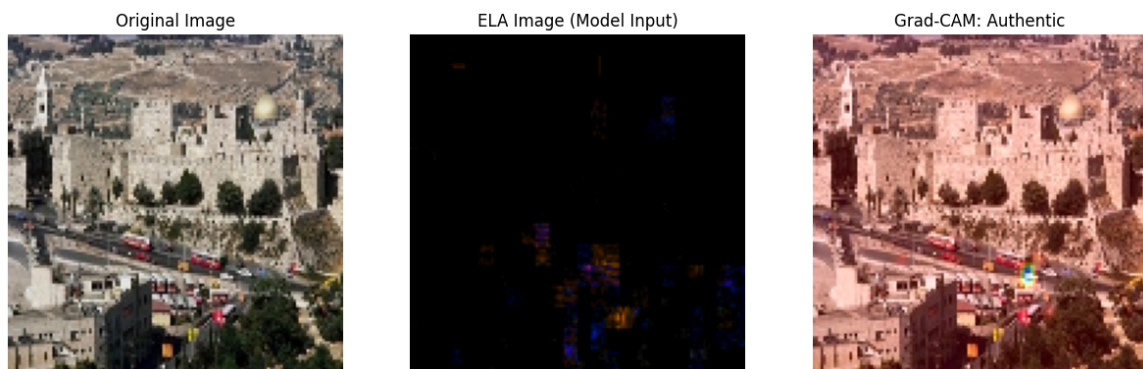


Figure 7: Ground Truth: Authentic, Predicted: Authentic (0.00)

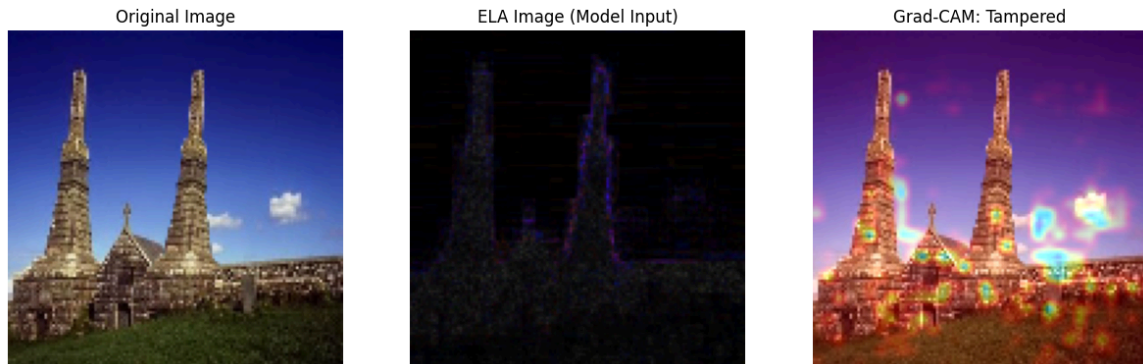


Figure 8: Ground Truth: Tampered, Predicted: Tampered (1.00)

These heatmaps visually indicate which parts of the image the model considers most important for its decision. For tampered images, Grad-CAM typically focuses on manipulated areas or their boundaries, providing intuitive evidence aligned with the goal of explainable digital forensics.

Chapter 5: Discussion

The results demonstrate that a relatively small CNN, when combined with ELA pre-processing, can achieve strong performance on CASIA v2.0 for binary image forgery detection. By converting all images into ELA representations, the model is encouraged to focus on compression inconsistencies rather than semantic content, which is beneficial for generalization across different image categories.

The architecture remains lightweight, with only three convolutional blocks and one hidden dense layer, making it suitable for deployment in resource-constrained environments such as mobile or edge devices. Training on a Google Colab GPU completes within a practical time frame due to the modest model size and fixed 128×128 input resolution.

Explainability is a key highlight of this project. Grad-CAM visualizations show that the network’s attention often aligns with tampered areas, providing human-understandable evidence of manipulation. When combined with frequency-domain insights, this supports a multi-perspective interpretation of how the model detects forgeries.

Chapter 6: Conclusion

This project successfully implements a complete pipeline for image forgery detection using a lightweight CNN and Error Level Analysis on the CASIA v2.0 dataset. The approach demonstrates that:

- ELA is an effective pre-processing technique for highlighting tampering artifacts.
- A compact CNN can achieve high classification performance while remaining computationally efficient.
- Grad-CAM adds interpretability by revealing which regions drive the model’s decisions, supporting explainable digital forensics.

Overall, the project provides a solid, reproducible work on lightweight, explainable image forgery detection using Deep Learning.